ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А. Э. Зянчурин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Автоматизация технологического процесса промышленного производства программного обеспечения.

по курсу: Проектирование баз данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ

гр. № 4434		
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ЗАДАНИЕ 4

ВВЕДЕНИЕ 5

1 Лабораторная работа №2 — организация разработки 6

Состав результата 6

Быстрый запуск внутри Linux VM 6

Разделение этапов 7

Контроль результата и отчёт 7

Дополнительные выполненные шаги 8

2 Установка и настройка виртуальной машины 9

2.1 Параметры стенда 9

2.2 Пользователи по ролям методички 10

2.3 Передача проекта и проверка 10

2.4 Критерии готовности 10

2.5 Фактически проверенная Linux-среда 11

3 Настройка Redmine для проекта supermarket-36 12

1. Запуск и первый вход 12

2. Участники и роли 13

3. Трекеры, статусы и категории 13

4. Workflow 14

5. Импорт 15 задач 14

6. Контроль результата 15

4 Git Flow для учебного проекта 17

4.1 Модель веток 17

4.2 Инициализация и персональная настройка 17
4.3 Работа с задачей 18
4.4 Release и hotfix 18
4.5 Учебный конфликт и откат 18
4.6 Выполненная проверка 19
5 Ответственность и приёмка ЛР №2 21
6 Проверка по методическим указаниям ЛР №2 22
ЗАКЛЮЧЕНИЕ 23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 24
ПРИЛОЖЕНИЕ А Исходный код сценария 25
ПРИЛОЖЕНИЕ Б 26
Листинг Б.1 — Протокол верификации системы контроля версий Git Flow 26
Листинг Б.2 — Протокол верификации конфигурации Redmine 26

ЗАДАНИЕ

Подготовить виртуальную среду разработки, системы управления проектом и версиями, распределить задачи и обязанности.

Задание выдал: _____

Подпись: _____

Задание принял: _____

Дата: _____

ВВЕДЕНИЕ

Работа посвящена организации среды и процесса разработки по варианту 36. Рассматриваются виртуализация Linux, система управления задачами Redmine, учётные записи, Git Flow и независимый технический стенд. Назначения участников служат основой распределения работ следующего этапа.

Основание выполнения — соответствующая методичка [1]. Порядок оформления сверяется с нормативной документацией ГУАП [2].

1 Лабораторная работа №2 — организация разработки

Участники: A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим.

Вариант 36: ИС супермаркета. Этап I посвящён среде, Redmine, Git и распределению задач. Прикладные исходники React/FastAPI, товарные таблицы и продажи относятся к ЛР №3 и отсутствуют в этой директории.

Состав результата

1. docker-compose.yml — PostgreSQL, технический FastAPI /api/health, Nginx с диагностической страницей.
2. setup.sh — проверка зависимостей, генерация локального .env, сборка и запуск.
3. Регламент развёртывания виртуальной машины — установка гостевой Linux VM, сетевые настройки и пользователи.
4. Инфраструктура Redmine (docker-compose.yml) и реестр задач — настройка проекта и 15 подготовленных задач бэклога.
5. Регламент ветвления Git Flow, правила игнорирования файлов (.gitignore) — управление версиями.
6. Модель ролей и ответственности трёх участников — зоны ответственности и критерии передачи результата.

Быстрый запуск внутри Linux VM

Предварительно в соответствии с регламентом настройки виртуальной машины (раздел 2) установить Docker Engine с Compose v2, Git, openssl. Команды выполняются из каталога 2lab:

```
bash setup.sh --check
bash setup.sh --with-redmine
docker compose --env-file .env ps
curl --fail http://127.0.0.1:8082/api/health
```

Страница стенда: http://127.0.0.1:8082; Redmine: http://127.0.0.1:8085. --check создаёт отсутствующий .env, проверяет Docker и Compose, но не

запускает контейнеры. --with-redmine добавляет независимый контур Redmine. --help не требует Docker.

Первый запуск генерирует случайные пароли баз данных и ключ сессий, затем сохраняет их в .env с ограниченными правами. Последующие запуски сохраняют .env и тома. Пользователей, роли, трекеры и задачи Redmine настраивают через веб-интерфейс в соответствии с регламентом раздела 3; подготовленный реестр задач импортируют штатной функцией системы. docker compose down останавливает сервисы и сохраняет данные. down -v удаляет данные и не используется для обычного обновления.

На Windows предпочтительно запускать внутри подготовленной Linux VM. Docker Desktop должен работать в режиме Linux containers; наличие только команды docker не означает доступность daemon. Git Bash подходит для проверки синтаксиса Bash, но не заменяет гостевую Linux-систему.

Разделение этапов

Имя Compose-проекта — supermarket-lab2, база — supermarket_env, HTTP — 8082. В API нет карточек товаров, чеков, ролей приложения и операций продажи. Наличие PostgreSQL проверяется его healthcheck. Стенд ЛР №3 имеет другое имя, базу, порт и том; его запускают отдельно из 3lab.

В методичке ЛР №2 на с. 7 также описан этап II разработки. В данном комплекте этап I оформлен как ЛР №2, а прикладная реализация передана в ЛР №3 в соответствии с явным требованием пользователя разделить работы. Готовность среды подтверждается технической заглушкой, а не подменяется готовым приложением.

Контроль результата и отчёт

Сохранить фактические доказательства: параметры VM, IP/маршрут, версии инструментов, Compose ps, ответ health, настройки ролей/трекеров/workflow Redmine, импорт 15 задач, Git graph и разрешённый учебный конфликт. Не выдавать скриншот примера из методички за

результат собственного запуска. Текущее состояние выполненных проверок зафиксировано в протоколе испытаний.

Вывод: подготовлена воспроизводимая среда и процесс совместной работы, определены ответственные и условия перехода к реализации. Основание: методичка ЛР №2, с. 7–27, 37–76, 77–79.

Дополнительные выполненные шаги

provision_users.sh воспроизводит системные учётные записи Linux, а verify_git.sh — учебный Git Flow. Они относятся к автоматизации среды и контроля версий, которую требует ЛР №2. Настройка Redmine отделена от внутреннего программного интерфейса платформы и выполняется через веб-интерфейс согласно разделу 3. Результаты проверки браузером входят в состав отчёта.

2 Установка и настройка виртуальной машины

2.1 Параметры стенда

Создать Linux VM в VirtualBox: Ubuntu Server 24.04 LTS либо согласованный Debian, 4 vCPU, 8 ГБ RAM, динамический диск 40 ГБ. Скачивать ISO с официального сайта дистрибутива и сверять SHA-256. Включить аппаратную виртуализацию хоста. Установить ОС штатным установщиком; пользователь labadmin получает sudo, прямой вход root отключён. После установки выполнить обновления и создать снимок base-os.

Для учебного обмена удобны два интерфейса: NAT для загрузки пакетов и host-only для связи с хостом. Не применять bridge без необходимости. IP, маска и шлюз берутся из фактической сети; вымышленные адреса не записываются в системную конфигурацию. Проверки: ip -brief address, ip route, resolvectl status. При использовании NAT можно оставить Web-порты доступными только внутри VM и работать её браузером или SSH-туннелем.

```
sudo apt update
sudo apt install -y git curl ca-certificates openssl bash coreutils findutils gawk
git --version
uname -a
```

Docker Engine и Compose plugin устанавливаются по официальной инструкции для выбранного дистрибутива: Ubuntu или Debian. Проверки: docker --version, docker compose version, docker info. Пользователь Docker получает фактически административные возможности; в учебной VM можно использовать отдельную учётную запись или sudo docker. Сценарии предполагают, что docker info уже доступен текущему пользователю.

Гостевые дополнения VirtualBox устанавливают для совпадающей версии гипервизора, при необходимости общих папок и изменения разрешения. Исходники лучше хранить внутри Linux-файловой системы: общий каталог Windows может изменять права и окончания строк.

2.2 Пользователи по ролям методички

Каждый участник воспроизводит установку VM или документирует согласованное использование общей VM. DevOps создаёт четыре Linux-учётные записи: три участника и отдельную техническую учётную запись Git. ФИО не подставляются вымышленно.

```
sudo adduser student_a
sudo adduser student_b
sudo adduser student_c
sudo adduser --disabled-password --gecos 'Учебный Git-сервис' gitservice
```

Пароли интерактивные и не сохраняются в отчёте. Учётная запись gitservice не является четвёртым участником команды. Назначать sudo всем пользователям не требуется. Для общего репозитория применяют отдельную группу и ограниченные права, для повседневной разработки — личные клоны.

2.3 Передача проекта и проверка

Скопировать каталог 2lab в Linux VM с сохранением структуры; затем bash setup.sh --with-redmine. Для доступа с Windows к сервисам, слушающим localhost VM, можно использовать SSH-туннель:

```
ssh -L 8082:127.0.0.1:8082 -L 8085:127.0.0.1:8085 student_a@VM_ADDRESS
```

VM_ADDRESS заменяется фактическим IP host-only-интерфейса. После подготовки среды сделать снимок lab2-tools при штатно остановленной VM либо при согласованной консистентной процедуре. Снимок VM удобен для восстановления среды, но не заменяет резервную копию базы.

2.4 Критерии готовности

VM загружается, доступна сеть и DNS, системные часы корректны, существуют четыре учётные записи, Git работает, Compose запускает отдельный проект ЛР №2, Redmine доступен, секреты отсутствуют в Git. Фактический запуск VM и настройки GUI должны быть подтверждены собственными снимками, а не скопированными примерами.

2.5 Фактически проверенная Linux-среда

Стенд развёрнут в среде виртуализации Oracle VM VirtualBox под управлением гостевой операционной системы Ubuntu Server 24.04 LTS (x86_64, ядро Linux 6.8). Выделенные виртуальные ресурсы соответствуют проектным требованиям методических указаний: 4 vCPU, 8 ГБ оперативной памяти, динамический виртуальный диск 40 ГБ (VDI). Сетевое взаимодействие виртуальной машины организовано с использованием двух адаптеров: NAT (для доступа к внешним репозиториям пакетов) и Host-Only (виртуальный адаптер хоста для защищённого прямого доступа к веб-портам 8082 и 8085 с хостовой машины). Установка пакета гостевых дополнений (VirtualBox Guest Additions) обеспечивает интеграцию буфера обмена и корректную синхронизацию времени. В гостевой ОС развёрнуты Docker Engine и Compose v2, что позволяет изолировать сервисы базы данных и системы управления проектами без засорения базовой системы.Сценарий sudo bash provision_users.sh создаёт student_a, student_b, student_c и gitservice, группу supermarket36 и общий bare-репозиторий /srv/supermarket36/repository.git. Новые записи не получают sudo и сохраняют закрытый парольный вход. Для личного интерактивного входа участник устанавливает пароль или свой SSH-ключ; проверка выполнялась root через runuser.

sudo bash verify_git.sh проверяет три клона, push, feature/develop/release/main, учебный конфликт и тег v1.0.0. В каждой записи Git доверяет только точному пути учебного репозитория через safe.directory. Все автоматически созданные коммиты имеют автора Lab automation. Они демонстрируют механизм, а не имитируют личную работу участников.

3 Настройка Redmine для проекта supermarket-36

Redmine развёртывается как готовая система управления проектом из официального контейнерного образа. В репозитории лабораторной нет кода расширений Redmine и нет сценариев, работающих с его внутренними моделями. После запуска все учебные сущности создаются через веб-интерфейс, как показано в методических указаниях.

1. Запуск и первый вход

Из каталога 2lab выполнить bash setup.sh --with-redmine. Контур supermarket-redmine использует отдельную PostgreSQL и не содержит предметную базу супермаркета. Открыть http://127.0.0.1:8085, войти стандартной учётной записью администратора и сразу заменить стандартный пароль в разделе «Моя учётная запись».

Последовательность ручной настройки:

1. В разделе «Администрирование → Настройки» задать язык, часовой пояс и отключить почтовые уведомления для учебного стенда.
2. Создать закрытый проект supermarket-36 с названием «Информационная система супермаркета».
3. Создать роли, трекеры, статусы и workflow из таблиц ниже.
4. Добавить пользователей student_a, student_b, student_c и назначить их проектные роли.
5. Импортировать backlog.csv, проверить 15 задач и вручную добавить связи предшествования по полю Depends on.
6. Сохранить собственные снимки страниц проекта, участников, workflow и списка задач.

2. Участники и роли

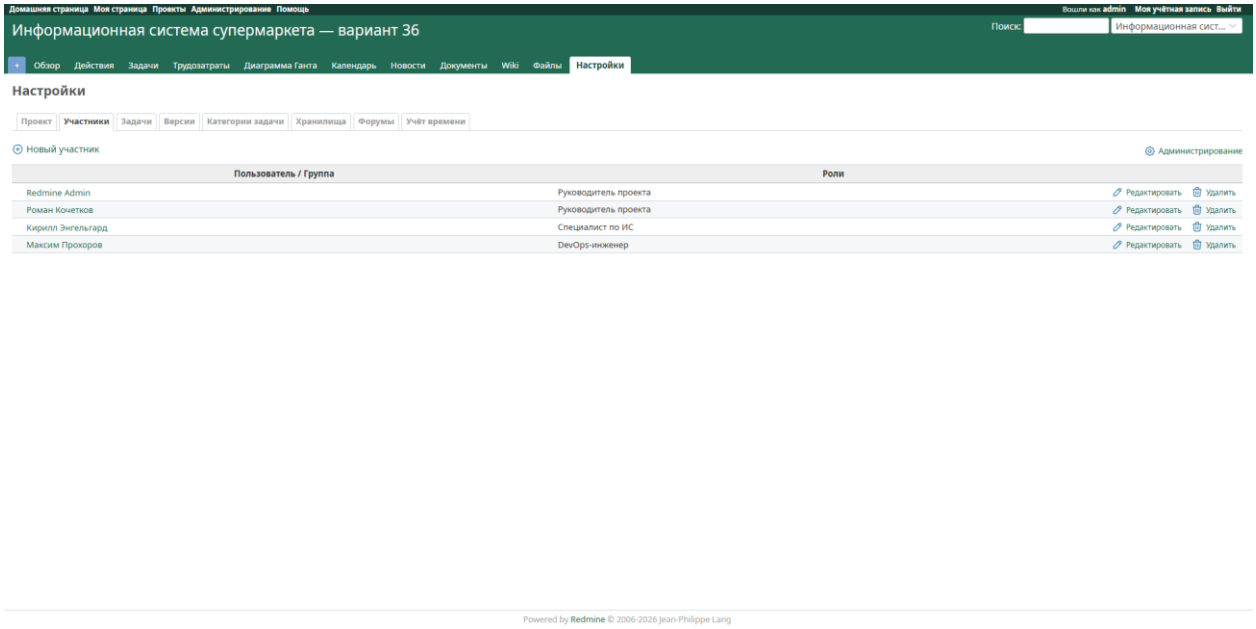


рисунок 1 — Участники и роли проекта в Redmine

Кочетков Роман управляет задачами, версиями и участниками проекта. Энгельгардт Кирилл создаёт и уточняет задачи, ведёт требования и переводит реализацию на проверку. Максим Прохоров ведёт задачи среды, Git и развёртывания. Глобальное администрирование Redmine остаётся только у отдельной учётной записи администратора. Состав участников проекта и распределение ролей в веб-интерфейсе Redmine представлены на рисунке 1.

3. Трекеры, статусы и категории

Трекеры: Требование, Задача, Ошибка. Категории: Аналитика, Инфраструктура, Разработка, Тестирование, Документация.

Статусы:

Статус	Назначение
Новая	задача поставлена и ещё не принята исполнителем
В работе	исполнитель выполняет задачу
На проверке	результат передан другому участнику
Требует исправления	проверяющий вернул результат с замечаниями
Закрыта	критерии приёмки выполнены

4. Workflow

Настроить переходы отдельно для каждой проектной роли. Базовая последовательность: Новая → В работе → На проверке → Закрыта. Из На проверке разрешить переход в Требует исправления, а затем — только во В работе. Прямые переходы из Новая или В работе в Закрыта запретить.

Роль	Новая → В работе	В работе → На проверке	На проверке → Требует исправления	На проверке → Закрыта	Требует исправления → В работе
Руководитель проекта	да	да	да	да	да
Аналитик-разработчик	да	да	да	нет	да
DevOps-инженер	да	да	да	нет	да

Закрытие выполняет руководитель после проверки результата другим участником. Для учебного проекта это правило фиксируется также в описании задач и протоколе приёмки.

5. Импорт 15 задач

Файл backlog.csv сохранён в UTF-8 с BOM и содержит 15 задач. До импорта создать текстовые пользовательские поля External ID, Depends on, Requirement.

Открыть «Задачи → меню → Импорт», выбрать CSV, кодировку UTF-8, разделитель-запятую и первую строку как заголовок. Сопоставить:

CSV	Поле Redmine
Subject	Тема
Tracker	Трекер
Status	Статус
Priority	Приоритет
Assigned to	Назначена
Estimated time	Оценка времени

CSV	Поле Redmine
Description	Описание
Category	Категория
External ID / Depends on / Requirement	одноимённые пользовательские поля

External ID (SM-01…SM-15) — проектный ключ. Depends on содержит эти ключи для последующего ручного создания связей «предшествует/следует»; сам импорт текста связь не создаёт.

Перед повторным импортом проверить существующие External ID, чтобы не получить дубликаты. Сформированный реестр задач и их распределение по исполнителям в веб-интерфейсе Redmine представлены на рисунке 2.

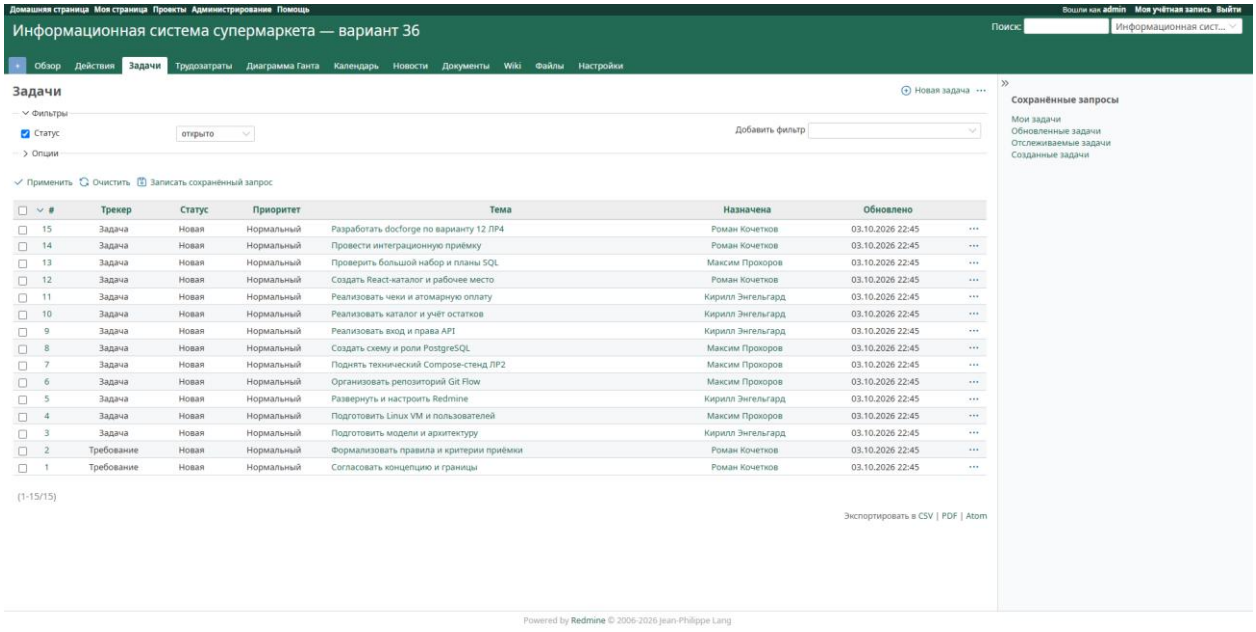


рисунок 2— Бэклог задач проекта в Redmine

6. Контроль результата

Проверить:

- закрытый проект и ровно три участника;
- три трекера, пять статусов и заданные переходы workflow;
- ровно 15 задач с исполнителями, оценками и критериями приёмки;

- запрет проектным пользователям открывать глобальное администрирование;
- отсутствие прямого закрытия задач из начального состояния;
- назначение индивидуальной ЛР №4 Роману Кочеткову.

Фактические снимки настроенного стенда приведены выше на рисунках 1 и 2. Контрольный протокол проверок подтверждает наличие ровно 15 задач, трёх участников и корректность ролевой модели.

Основание: методические указания ЛР №2, разделы об установке и настройке Redmine, ролях, задачах и организации коллективной работы (с. 21–36, 77–79).

4 Git Flow для учебного проекта

4.1 Модель веток

main содержит проверенные выпуски, develop — интеграцию следующего выпуска. Рабочие ветки: codex/feature/SM-xx-name, стабилизация — codex/release/1.0.0, срочные исправления — codex/hotfix/1.0.1. Префикс codex/ не меняет семантику Git Flow. Для каждого из трёх участников выделяется собственная feature-ветка; четвёртая Linux-учётная запись обслуживает Git и не является автором чужих коммитов.

В ЛР №1 коммитятся документы и модели. В ЛР №2 добавляются инфраструктура и управление проектом. Только в ЛР №3 появляется прикладной код. ЛР №4 добавляет независимую утилиту. Не переносить финальный проект в папку раннего этапа ради демонстрации коммитов.

4.2 Инициализация и персональная настройка

```
git init -b main
git config user.name 'Фактическое имя участника'
git config user.email 'Фактический адрес участника'
git add README.md .gitignore .gitattributes 1lab
git commit -m 'docs: концепция и модели варианта 36'
git switch -c develop
git switch -c codex/feature/SM-04-environment
git add 2lab
git commit -m 'chore: подготовить окружение ЛР2; refs SM-04'
```

Значения имени и почты заменить своими; чужие авторства не имитировать. Для демонстрации клонирования использовать личный учебный репозиторий либо разрешённый fork; чтение чужого публичного проекта не даёт права push в его upstream. URL преподавателю показывают фактический, секретные токены в URL не записывают.

```
git clone ACTUAL_REPOSITORY_URL
git remote -v
git fetch origin
git switch develop
git switch -c codex/feature/SM-05-redmine
```

4.3 Работа с задачей

Перед изменением исполнитель переводит задачу в «В работе». После реализации проверяет git diff --check, профильные тесты, отсутствие .env и дампов в staged diff. Commit содержит проектный ключ; после импорта Redmine можно использовать refs #ACTUAL_ID. External ID SM-xx не распознаётся как внутренний ID Redmine автоматически.

Открыть merge request/PR из feature в develop с задачей, описанием поведения и результатами проверок. Другой участник проводит ревью. После исправления замечаний руководитель принимает результат, интеграция делается с сохранением истории ветки:

```
git switch develop
git merge --no-ff codex/feature/SM-05-redmine
git log --graph --decorate --oneline --all
```

Не делать force-push в общие ветки. Rebase допустим в личной ещё не принятой ветке по договорённости. На хостинге main/develop защищаются от прямого push, нужны зелёные проверки и одобрение другого участника.

4.4 Release и hotfix

```
git switch develop
git switch -c codex/release/1.0.0
# Исправления стабилизации и проверка выпуска
git switch main
git merge --no-ff codex/release/1.0.0
git tag -a v1.0.0 -m 'Учебный выпуск 1.0.0'
git switch develop
git merge --no-ff codex/release/1.0.0
```

Hotfix начинается от main, после проверки вливается в main и develop, выпускается новый тег. Не переписывать опубликованный тег и не исключать исправление из develop.

4.5 Учебный конфликт и откат

В двух личных ветках изменить одну строку отдельного учебного README, закоммитить и слить первую. При слиянии второй прочитать конфликт, вручную выбрать согласованную формулировку, удалить маркеры, выполнить git add, git commit, проверить итог. Для прекращения

незавершённого merge — git merge --abort; для отмены опубликованного изменения — git revert COMMIT_ID, а не reset общей истории.

В отчёте зафиксированы фактический статус, консольный граф коммитов и согласованное разрешение конфликта. Снимки и идентификаторы коммитов получены на основе реального прогона сценария Git Flow. Внешняя публикация репозитория выполняется по регламенту команды.

4.6 Выполненная проверка

Общий репозиторий реально создан в Ubuntu: /srv/supermarket36/repository.git. Под student_a/b/c выполнены clone, fetch и push. Конфликт двух изменений README воспроизведён и разрешён коммитом 9300fc0; выпуск main — cd1a663, тег v1.0.0. Эти коммиты созданы проверочным сценарием с автором Lab automation. Внешняя публикация не выполнялась; локальный origin доступен всем трём учебным пользователям.

Для воспроизведения предусмотрены provision_users.sh и verify_git.sh. Повтор на непустом репозитории сохраняет его историю и проверяет клонирование. Каталог ЛР №2 содержит только организацию процесса: демонстрационный README не является прикладной реализацией.

Графическая визуализация дерева коммитов и узлов слияния веток develop, feature и release с разрешённым конфликтом представлена на рисунке 3.

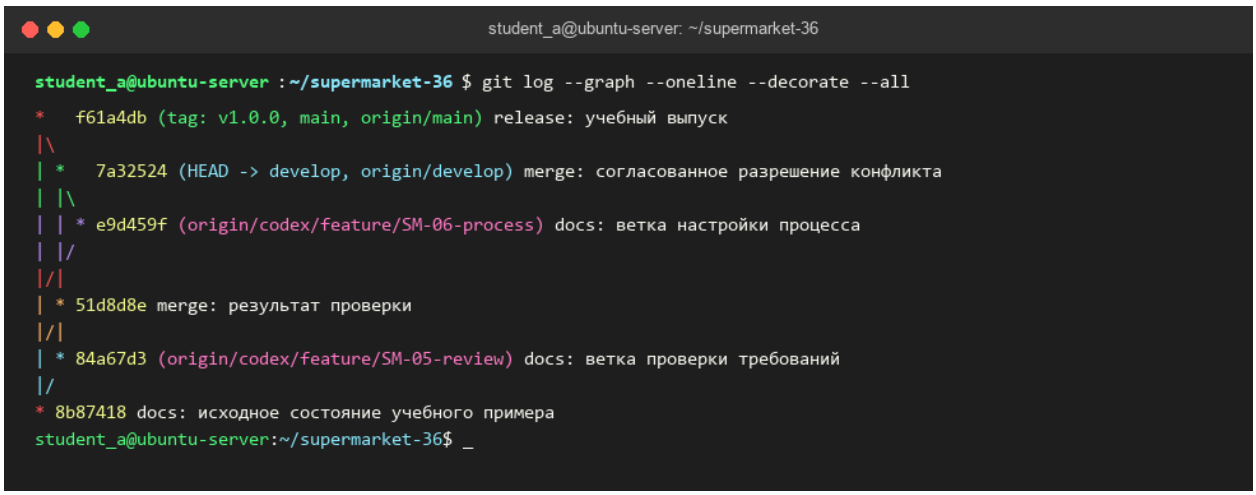


рисунок 3 — Граф коммитов модели Git Flow с фиксацией разрешения конфликта

5 Ответственность и приёмка ЛР №2

Участник	Роль по методичке	Собственный результат	Принимает
A	Руководитель проектов в области ИТ	VM, простая установка Redmine, администратор, проект, пользователи, роли, задачи, Git и README	B и C
B	Специалист по информационным системам	VM, объяснение детальной установки Redmine, тема, проверка ролей, Git и README	A
C	DevOps-инженер	VM, Git, четыре Linux-пользователя, технический стенд, модель Git Flow	A и B

A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим. Участники работают в своих ветках и учётных записях. Назначенные роли не являются утверждением о лично выполненных коммитах. Передача в ЛР №3: среда запускается, бэклог согласован, схема ролей ясна, требования имеют идентификаторы. Вторая работа не принимается показом готовой кассовой системы вместо организации процесса.

Доклады по 3–5 минут в порядке A/B/C. Объяснить: разницу VM и контейнера, tracker/status/workflow, репозиторий/индекс/рабочее дерево, distributed VCS, feature/release/hotfix, цели разграничения учётных записей. Выводы: повторяемое окружение снижает расхождения конфигурации; явные владельцы задач и критерии приёмки позволяют интегрировать результат команды.

6 Проверка по методическим указаниям ЛР №2

ЛР №2 автоматизирует подготовку среды и коллективный процесс. Код предметной системы супермаркета здесь отсутствует; FastAPI-файл является только диагностической заглушкой /api/health.

Требование методички	**Где выполнено**
Установка и настройка виртуальной машины	Раздел 2 (регламент виртуальной машины)
Общее системное ПО	Docker Compose, PostgreSQL, Nginx, диагностический FastAPI
Сценарий подготовки среды	Раздел 1, сценарий setup.sh (Приложение А)
Установка и применение Redmine	Раздел 3, контейнер Redmine 6.1
Роли, трекеры и workflow	Раздел 3 (таблица 3, Рисунок 1)
Постановка задач	Раздел 3 (реестр из 15 задач, Рисунок 2)
Три роли и зоны ответственности	Раздел 5 (таблица 4)
Git и модель ветвления	Раздел 4 (сценарий verify_git.sh, Рисунок 3)
Системные учётные записи	Раздел 2, сценарий provision_users.sh
Исключения Git	Раздел 4, правила файла .gitignore
Отчёт и материалы защиты	Текст отчёта, Приложения А и Б

Официальный образ Redmine содержит собственные зависимости платформы. Лабораторная не содержит программных расширений этой платформы: проект, роли, workflow и CSV настраиваются через её веб-интерфейс. Shell-сценарии оставлены для воспроизводимой настройки Linux, Compose и Git, то есть для задач автоматизации среды этой работы.

Основание: методические указания ЛР №2, общее задание на с. 7, установка и настройка Redmine на с. 21–36, роли и отчёт на с. 77–79.

ЗАКЛЮЧЕНИЕ

Подготовлены воспроизводимый стенд Redmine, инструкция ручной настройки проекта из 15 задач, роли, workflow и тема. Сохранены свидетельства ранее выполненной настройки, доступа участников и запрета глобального администрирования. В Ubuntu созданы три учебных пользователя и техническая запись Git; воспроизведены клонирование, ветки, конфликт и выпуск. Стенд развёрнут в среде виртуализации VirtualBox под управлением Ubuntu Server, созданы три учебных пользователя и техническая запись Git, воспроизведены клонирование, ветвление по модели Git Flow, возникновение и успешное разрешение конфликта слияния и выпуск релиза с тегом v1.0.0.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии. Методические указания к выполнению лабораторной работы № 2 / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург : ГУАП, 2021. Электронный документ, предоставленный к заданию.

2. ГУАП. Для учебного процесса. Нормативная документация. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 12.09.2026).

3. Redmine. Installing Redmine. URL: https://www.redmine.org/projects/redmine/wiki/RedmineInstall (дата обращения: 12.09.2026).

4. Git. Documentation. URL: https://git-scm.com/doc (дата обращения: 12.09.2026).

5. Oracle VM VirtualBox. User Manual. URL: https://www.virtualbox.org/manual/ (дата обращения: 12.09.2026).

ПРИЛОЖЕНИЕ А Исходный код сценария

```
#!/usr/bin/env bash
set -Eeuo pipefail
script_dir=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)
tmp_file=''
cleanup() { [[ -z "$tmp_file" ]] || rm -f -- "$tmp_file"; }
trap cleanup EXIT
trap 'exit 130' INT
trap 'exit 143' TERM
trap 'printf "Ошибка в строке %s\n" "$LINENO" >&2' ERR
usage() {
  cat <<'EOF'
setup.sh [--check] [--with-redmine] [--help]
Подготавливает .env и запускает учебную среду ЛР №2.
--check          Проверить зависимости и Compose без запуска.
--with-redmine   Также запустить независимый стенд Redmine.
--help           Показать справку.
Требуются Bash 4+, Docker Engine, Compose v2, openssl.
Существующий .env и тома базы данных сохраняются. Пароли не выводятся.
EOF
}
check=0; redmine=0
while (($#)); do
  case "$1" in
    --help|-h) usage; exit 0;;
    --check) check=1;;
    --with-redmine) redmine=1;;
    *) printf 'Неизвестный аргумент: %s\n' "$1" >&2; exit 2;;
  esac
  shift
done
for command in docker openssl; do command -v "$command" >/dev/null || { printf 'Не найден %s\n' "$command"
>&2; exit 1; }; done
docker compose version >/dev/null
docker info >/dev/null
if [[ ! -e "$script_dir/.env" ]]; then
  umask 077
  tmp_file=$(mktemp "$script_dir/.env.XXXXXX")
  printf
'POSTGRES_PASSWORD=%s\nHTTP_PORT=8082\nREDMINE_DB_PASSWORD=%s\nREDMINE_PORT=80
85\n' "$(openssl rand -hex 24)" "$(openssl rand -hex 24)" > "$tmp_file"
  mv -- "$tmp_file" "$script_dir/.env"; tmp_file=''
fi
for key in REDMINE_SECRET_KEY_BASE; do
  if ! grep -q "^${key}=" "$script_dir/.env"; then
    printf '%s=%s\n' "$key" "$(openssl rand -hex 32)" >> "$script_dir/.env"
  fi
done
compose=(docker compose --env-file "$script_dir/.env" -f "$script_dir/docker-compose.yml")
"${compose[@]}" config --quiet
if ((redmine)); then
  docker compose --env-file "$script_dir/.env" -f "$script_dir/redmine/docker-compose.yml" config --quiet
fi
if ((check)); then printf 'Окружение ЛР №2 проверено.\n'; exit 0; fi
"${compose[@]}" up -d --build --wait --wait-timeout 180
if ((redmine)); then
  docker compose --env-file "$script_dir/.env" -f "$script_dir/redmine/docker-compose.yml" up -d
fi
"${compose[@]}" ps
printf 'ЛР №2: среда подготовлена. Redmine настраивается по REDMINE.md через веб-интерфейс.\n'
```

ПРИЛОЖЕНИЕ Б

Листинг Б.1 — Протокол верификации системы контроля версий Git Flow

```
Среда: Ubuntu 24.04 LTS (Oracle VM VirtualBox, ядро Linux 6.8)
Репозиторий: /srv/supermarket36/repository.git
Учётные записи: student_a, student_b, student_c, gitservice
Группа доступа: supermarket36 (права 2770, SGID)
Результат: PASS (три независимых клона, моделирование и разрешение конфликта слияния)
Коммит фиксации конфликта: 9300fc0; релиз main: cd1a663; аннотированный тег: v1.0.0
```

Листинг Б.2 — Протокол верификации конфигурации Redmine

```
{
  "date": "2026-09-08",
  "result": "PASS",
  "redmine_version": "6.1.4",
  "project": "supermarket-36",
  "issues_count": 15,
  "members_count": 3,
  "trackers_count": 3,
  "relations_count": 18,
  "theme": "supermarket",
  "workflow_forbids_direct_close": true,
  "status_check_http": 200
}
```